

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет прикладної математики та інформатики
Кафедра математичного моделювання соціально-економічних процесів

Звіт

Лабораторна робота №2

Виконав:

студент групи Пма-42

Думанський Юрій

Перевірив:

Професор

Недашковський М. О.

Львів-2026

Тема:

Розкриття невизначеності протидії двох суб'єктів

Мета:

Визначити гарантовані результати двох суб'єктів різними методами, знайти множину Парето та обчислити оптимальні компромісні значення параметрів.

Завдання:

При заданих цільових функціях f_{12} та f_{21} визначити гарантовані результати f_{12}^* , f_{21}^* табличним, графічним і класичними методами. Знайти множину Парето з умови $f_{12} \geq f'_{12} f_{21} \geq f_{21}$. Визначити оптимальні значення x_1^* і x_2^* за яких $\delta_i = |f_i(x_1^*, x_2^*) - f_i^*|$ набуває мінімального значення $\delta \rightarrow 0$. Варіант 9.

Хід роботи:

- Розв'язання задачі розпочато з аналітичного методу знаходження гарантованих результатів.

Для першого суб'єкта знаходимо максимум. Цільова функція: $f_{12} = 6x_1^2 - 12x_1 + 4x_2^2 + 8x_2 + 40$. Знаходимо мінімум по x_2 : похідна $8x_2 + 8 = 0 \Rightarrow x_2 = -1$. Підставляємо це значення: $\min f_{12} = 6x_1^2 - 12x_1 + 36$. Максимум цієї параболи на інтервалі $[-2, 2]$ знаходиться на лівому краю: при $x_1 = -2$ значення дорівнює 84. Отже, $f_{12}^* = 84$.

Для другого суб'єкта цільова функція: $f_{21} = -8x_1^2 + 16x_1 - 3x_2^2 + 6x_2 + 50$. Знаходимо мінімум по x_1 . Парабола гілками вниз має мінімум на краях інтервалу $[-2, 2]$: при $x_1 = -2$ значення -64 , при $x_1 = 2$ значення 0. Найгірший випадок $x_1 = -2$. Підставляємо: $\min f_{21} = -3x_2^2 + 6x_2 - 14$. Шукаємо максимум по x_2 : похідна $-6x_2 + 6 = 0 \Rightarrow x_2 = 1$. Підставляємо $x_2 = 1$ і отримуємо $f_{21}^* = -11$.

- Для програмної реалізації задачі використано мову Python з бібліотеками numpy для обчислень, matplotlib для графіків та tkinter для інтерфейсу.
- Табличний метод реалізовано через генерацію двовимірної сітки значень x_1 та x_2 з кроком 0.01. Обчислено матриці значень функцій f_{12} та f_{21} . Для кожної функції знайдено мінімуми по відповідних осях очікуваних дій опонента, після чого обрано максимальні значення, що підтвердило ручні розрахунки.

- Множину Парето знайдено шляхом накладання умов $f_{12} \geq \alpha f_{21} \geq -1$ на обчислені матриці. Серед відфільтрованих точок знайдено таку пару (x_1^*, x_2^*) , для якої сума відхилень δ від гарантованих результатів є найменшою.

Результати:

Аналітичні розрахунки повністю підтвердилися результатами роботи програми. Гарантовані результати склали: $f_{12}^* = 84$ (при $x_1 = -2$) та $f_{21}^* = -11$ (при $x_2 = 1$). Програма відфільтрувала точки, що утворюють множину Парето, та побудувала графіки простору критеріїв і простору рішень. Визначено компромісну точку з мінімальним відхиленням δ .

Висновки:

В ході роботи було досліджено задачу розкриття невизначеності при протидії двох суб'єктів. Використання аналітичного та табличного методів дало змогу точно визначити гарантовані результати. Програмна реалізація дозволила автоматизувати пошук множини Парето та знайти оптимальне компромісне рішення.

Додаток 1. Код програми.

```
import tkinter as tk
from tkinter import ttk, messagebox, scrolledtext
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from scipy.optimize import minimize

class GameTheoryOptimizerApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Розкриття невизначеності протидії двох суб'єктів")
        self.root.geometry("1200x850")

        input_frame = ttk.LabelFrame(root, text="Вхідні параметри", padding=10)
        input_frame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=5)

        font_main = ('Arial', 10)

        #Рядок 1: Функції
        ttk.Label(input_frame, text="Функція f12(x1, x2):", font=font_main,
        foreground="darkblue").grid(row=0, column=0, sticky="e", padx=5, pady=5)
        self.entry_f12 = ttk.Entry(input_frame, width=35, font=font_main)
        self.entry_f12.insert(0, "6*x1**2 - 12*x1 + 4*x2**2 + 8*x2 + 40")
        self.entry_f12.grid(row=0, column=1, columnspan=2, sticky="w", padx=5, pady=5)

        ttk.Label(input_frame, text="Функція f21(x1, x2):", font=font_main, foreground="darkred").grid(row=0,
        column=3, sticky="e", padx=5, pady=5)
        self.entry_f21 = ttk.Entry(input_frame, width=35, font=font_main)
        self.entry_f21.insert(0, "-8*x1**2 + 16*x1 - 3*x2**2 + 6*x2 + 50")
        self.entry_f21.grid(row=0, column=4, columnspan=2, sticky="w", padx=5, pady=5)

        #Рядок 2: Межі x1 та x2
        ttk.Label(input_frame, text="Межі x1 (min, max):", font=font_main).grid(row=1, column=0, sticky="e",
        padx=5, pady=5)
        self.entry_x1_min = ttk.Entry(input_frame, width=10, font=font_main)
        self.entry_x1_min.insert(0, "-2")
        self.entry_x1_min.grid(row=1, column=1, sticky="w", padx=5, pady=5)

        self.entry_x1_max = ttk.Entry(input_frame, width=10, font=font_main)
        self.entry_x1_max.insert(0, "2")
        self.entry_x1_max.grid(row=1, column=2, sticky="w", padx=5, pady=5)

        ttk.Label(input_frame, text="Межі x2 (min, max):", font=font_main).grid(row=1, column=3, sticky="e",
        padx=5, pady=5)
        self.entry_x2_min = ttk.Entry(input_frame, width=10, font=font_main)
        self.entry_x2_min.insert(0, "-2")
        self.entry_x2_min.grid(row=1, column=4, sticky="w", padx=5, pady=5)

        self.entry_x2_max = ttk.Entry(input_frame, width=10, font=font_main)
        self.entry_x2_max.insert(0, "2")
        self.entry_x2_max.grid(row=1, column=5, sticky="w", padx=5, pady=5)

        # Рядок 3: Кроки та кнопка
```

```

    ttk.Label(input_frame, text="Крок сітки:", font=font_main).grid(row=2, column=0, sticky="e", padx=5,
pady=5)
    self.entry_step = ttk.Entry(input_frame, width=10, font=font_main)
    self.entry_step.insert(0, "0.01")
    self.entry_step.grid(row=2, column=1, sticky="w", padx=5, pady=5)

    self.btn_calc = ttk.Button(input_frame, text="Розрахувати та Побудувати", command=self.calculate)
    self.btn_calc.grid(row=2, column=3, colspan=3, sticky="ew", padx=10, pady=5)

    # Текстове поле
    self.result_text = scrolledtext.ScrolledText(root, height=10, font=('Consolas', 10), bg="#f8f9fa")
    self.result_text.pack(side=tk.TOP, fill=tk.X, padx=10, pady=5)

    # Область для графіків
    plot_frame = ttk.Frame(root)
    plot_frame.pack(side=tk.BOTTOM, fill=tk.BOTH, expand=True, padx=10, pady=5)

    self.fig, (self.ax1, self.ax2) = plt.subplots(1, 2, figsize=(12, 5))
    self.fig.tight_layout(pad=4.0)
    self.canvas = FigureCanvasTkAgg(self.fig, master=plot_frame)
    self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

def parse_function(self, func_str, x1_val, x2_val):
    try:
        func_str = func_str.replace("^", "**")
        allowed_names = {
            "x1": x1_val, "x2": x2_val,
            "sin": np.sin, "cos": np.cos, "tan": np.tan,
            "exp": np.exp, "sqrt": np.sqrt, "log": np.log, "lg": np.log10,
            "abs": np.abs, "pi": np.pi
        }
        return eval(func_str, {"__builtins__": {}}, allowed_names)
    except Exception as e:
        raise ValueError(f"Помилка у формулі: {e}")

def classical_maximin(self, func_str, var_max, var_min, bounds_max, bounds_min):
    # Внутрішня мінімізація
    def objective_min(v_min, v_max_val):
        kwargs = {var_max: v_max_val, var_min: v_min[0]}
        return self.parse_function(func_str, kwargs.get('x1'), kwargs.get('x2'))

    # Зовнішня максимізація (через мінімізацію від'ємного значення)
    def objective_max(v_max):
        res_min = minimize(objective_min, x0=[(bounds_min[0]+bounds_min[1])/2], args=(v_max[0],),
bounds=[bounds_min])
        return -res_min.fun

    res_max = minimize(objective_max, x0=[(bounds_max[0]+bounds_max[1])/2], bounds=[bounds_max])
    best_v_max = res_max.x[0]
    best_guaranteed_val = -res_max.fun
    return best_v_max, best_guaranteed_val

def calculate(self):
    try:
        # Зчитування даних
        str_f12 = self.entry_f12.get()

```

```

str_f21 = self.entry_f21.get()

x1_min = float(self.entry_x1_min.get())
x1_max = float(self.entry_x1_max.get())
x2_min = float(self.entry_x2_min.get())
x2_max = float(self.entry_x2_max.get())
step = float(self.entry_step.get())

if x1_min >= x1_max or x2_min >= x2_max:
    raise ValueError("Початкові межі мають бути меншими за кінцеві.")
if step <= 0:
    raise ValueError("Крок сітки має бути більшим за 0.")

# Створення сіток
x1_vals = np.arange(x1_min, x1_max + step, step)
x2_vals = np.arange(x2_min, x2_max + step, step)
X1, X2 = np.meshgrid(x1_vals, x2_vals, indexing='ij')

# Обчислення функцій
F12 = self.parse_function(str_f12, X1, X2)
F21 = self.parse_function(str_f21, X1, X2)

# 1. Табличний метод
# Для Суб'єкта 1
min_f12_over_x2 = np.min(F12, axis=1)
idx_max_f12 = np.argmax(min_f12_over_x2)
f12_star_tab = min_f12_over_x2[idx_max_f12]
x1_star_tab = x1_vals[idx_max_f12]

# Для Суб'єкта 2
min_f21_over_x1 = np.min(F21, axis=0)
idx_max_f21 = np.argmax(min_f21_over_x1)
f21_star_tab = min_f21_over_x1[idx_max_f21]
x2_star_tab = x2_vals[idx_max_f21]

# 2. Класичний метод
x1_star_cls, f12_star_cls = self.classical_maximin(str_f12, 'x1', 'x2', (x1_min, x1_max), (x2_min,
x2_max))
x2_star_cls, f21_star_cls = self.classical_maximin(str_f21, 'x2', 'x1', (x2_min, x2_max), (x1_min,
x1_max))

# 3. Множина Парето та Оптимуми
pareto_mask = (F12 >= f12_star_tab) & (F21 >= f21_star_tab)

opt_x1, opt_x2 = None, None
min_delta = float('inf')
f12_opt, f21_opt = None, None

if np.any(pareto_mask):
    for i in range(len(x1_vals)):
        for j in range(len(x2_vals)):
            if pareto_mask[i, j]:
                # Обчислюємо дельту
                delta = abs(F12[i, j] - f12_star_tab) + abs(F21[i, j] - f21_star_tab)
                if delta < min_delta:
                    min_delta = delta

```

```

        opt_x1, opt_x2 = x1_vals[i], x2_vals[j]
        f12_opt, f21_opt = F12[i, j], F21[i, j]

# Вивід результатів
self.result_text.delete(1.0, tk.END)
self.result_text.insert(tk.END, "===== 1. ГАРАНТОВАНИ РЕЗУЛЬТАТИ =====\n")
self.result_text.insert(tk.END, f"[Табличний] f12*: {f12_star_tab:.4f} (при x1 = {x1_star_tab:.4f})\n")
self.result_text.insert(tk.END, f"[Табличний] f21*: {f21_star_tab:.4f} (при x2 = {x2_star_tab:.4f})\n")
self.result_text.insert(tk.END, f"-----\n")
self.result_text.insert(tk.END, f"[Аналітичний] f12*: {f12_star_cls:.4f} (при x1 = {x1_star_cls:.4f})\n")
self.result_text.insert(tk.END, f"[Аналітичний] f21*: {f21_star_cls:.4f} (при x2 = {x2_star_cls:.4f})\n\n")

self.result_text.insert(tk.END, "===== 2. МНОЖИНА ПАРЕТО ТА КОМПРОМІС =====\n")
if np.any(pareto_mask):
    num_pareto = np.sum(pareto_mask)
    self.result_text.insert(tk.END, f"Умов Парето (f >= f*) дотримуються {num_pareto} точок.\n")
    self.result_text.insert(tk.END, f"--> Оптимальна компромісна точка ( $\Delta \rightarrow 0$ ): x1* = {opt_x1:.4f}, x2* = {opt_x2:.4f}\n")
    self.result_text.insert(tk.END, f"--> Значення функцій: f12 = {f12_opt:.4f}, f21 = {f21_opt:.4f}\n")
    self.result_text.insert(tk.END, f"--> Мінімальне відхилення  $\Delta$  = {min_delta:.4f}\n")
else:
    self.result_text.insert(tk.END, "Множина Парето порожня. Немає точок, що задовольняють умови.\n")

# Графіки
self.ax1.clear()
self.ax2.clear()

# Графік 1: f12
# Малюємо проріджені лінії для наочності
step_draw = max(1, len(x2_vals)//10)
for j in range(0, len(x2_vals), step_draw):
    self.ax1.plot(x1_vals, F12[:, j], color='teal', alpha=0.25, linestyle='-.')

# Обвідна лінія
self.ax1.plot(x1_vals, min_f12_over_x2, color='darkblue', linewidth=2.5, label='Нижня обвідна (min x2)')
self.ax1.scatter([x1_star_tab], [f12_star_tab], color='magenta', s=70, zorder=5, label=f'Maximin f12*={f12_star_tab:.1f}')

self.ax1.set_title("Суб'єкт 1: f12 (гарантований результат)", fontweight='bold')
self.ax1.set_xlabel("Значення x1")
self.ax1.set_ylabel("f12(x1, x2)")
self.ax1.legend(loc="lower right")
self.ax1.grid(True, linestyle=':', alpha=0.7)

# Графік 2: f21
step_draw_2 = max(1, len(x1_vals)//10)
for i in range(0, len(x1_vals), step_draw_2):
    self.ax2.plot(x2_vals, F21[i, :], color='sienna', alpha=0.25, linestyle='-.')

self.ax2.plot(x2_vals, min_f21_over_x1, color='darkred', linewidth=2.5, label='Нижня обвідна (min x1)')
self.ax2.scatter([x2_star_tab], [f21_star_tab], color='magenta', s=70, zorder=5, label=f'Maximin f21*={f21_star_tab:.1f}')

```

```
self.ax2.set_title("Суб'єкт 2: f21 (гарантований результат)", fontweight='bold')
self.ax2.set_xlabel("Значення x2")
self.ax2.set_ylabel("f21(x1, x2)")
self.ax2.legend(loc="lower right")
self.ax2.grid(True, linestyle=':', alpha=0.7)

self.fig.tight_layout()
self.canvas.draw()

except Exception as e:
    messagebox.showerror("Помилка", str(e))

if __name__ == "__main__":
    root = tk.Tk()
    app = GameTheoryOptimizerApp(root)
    root.mainloop()
```